EXHIBITION • VIVA • LIVE DEMONSTRATION

PlantPal

A practical guide to explaining, testing and defending your project



IoT-Enabled Smart Plant Monitoring and Voice Assistance System

A working ESP32 prototype centred on one Tulsi plant (Ocimum tenuiflorum). It senses the plant environment, interprets readings locally, gives OLED and voice feedback, and extends monitoring through a browser dashboard.

Team: Aashutosh Mishra (SDIT060A) • Akshat Chheda (SDIT011A) • Vivek Dubey (SDIT016A)
Guides: Mr. Aryan Mourya • Mr. Sumit Singh
Programme: S.Y. B.Sc. IT • K. E. S. Shroff College (Autonomous) • 2026–27

Important: the report records earlier subsystem tests, but the final assembled plant-and-sensor logic still needs a complete pre-exhibition run. Do not describe an unverified test as passed.

1. Explain PlantPal in 30–60 seconds

“PlantPal is an IoT-enabled smart plant monitoring and voice assistance system built around an ESP32 and a Tulsi plant. A capacitive soil sensor measures a relative soil-moisture value, the BH1750 measures ambient light, the DHT11 measures temperature and humidity, and a touch module provides user input. The ESP32 interprets the readings and presents local feedback through an OLED and a DFPlayer Mini speaker. Over Wi-Fi, it sends telemetry to a Flask API backed by SQLite, where a web dashboard displays readings and queues remote commands. Watering is intentionally human-controlled, and core local monitoring is designed to continue if the cloud is unavailable.”

Problem and design goal

• Plant care can rely on occasional visual inspection, which may miss changes in soil or environmental conditions.

• The aim is to make sensor readings easier to understand through one local device plus remote visibility.

• Design sequence: Sense → Validate → Classify → Communicate → Synchronise → Poll for commands.

What PlantPal is — and is not

• It is: an embedded systems + IoT integration prototype with local sensing, user interaction, stored voice prompts and a cloud dashboard.

• It is not: an autonomous irrigation system, plant-disease detector or laboratory-grade plant-health instrument.

• The health score and moisture percentage are project-specific indicators; they depend on calibration and chosen thresholds.

Remember this one-liner

PlantPal senses the environment, interprets the readings, communicates locally, and synchronises useful information to the cloud.

2. Hardware, pin map and purpose

Part	Connection / interface	Purpose
ESP32 Dev Module	Central controller; 2.4 GHz Wi-Fi	Reads sensors, runs logic, manages local UI and cloud
OLED SSD1306	SDA GPIO21 • SCL GPIO22 • I²C 0x3C	Local status and sensor pages
BH1750 / GY-302	SDA GPIO21 • SCL GPIO22 • 3.3V + GND	Ambient illuminance in lux
Capacitive soil V2.0	AOUT GPIO34 • 3.3V + GND	Relative soil-moisture raw reading
DHT11	DATA GPIO4 • 3.3V + GND	Temperature and relative humidity
TTP223 touch	SIG GPIO27 • 3.3V + GND	User input / gestures
Buzzer	+ GPIO25 • − GND	Short local alert / feedback
DFPlayer Mini	TX → ESP32 GPIO16 (RX2); RX ← GPIO17 (TX2); proper 5V supply; common GND	Plays stored MP3 voice prompts
Speaker	DFPlayer SPK1 and SPK2; neither terminal to GND	Audible output

Electrical points to remember

- OLED and BH1750 share I²C on GPIO21/22; that is normal when their addresses do not conflict.
- GPIO34 is input-only and ADC-capable, suitable for the soil sensor’s analog output.
- Keep sensor logic at 3.3 V. Power DFPlayer from a proper 5 V supply and connect its ground to ESP32 ground.
- VN is GPIO39, not a 5 V output. Never use it to power the DFPlayer.
- Keep the soil sensor connector above soil, DHT11 exposed to air, and BH1750 unobstructed.

3. Software and system architecture

Local path

Sensors → ESP32 acquisition and validation → plant-condition states / composite score → OLED, touch handling, audio and buzzer.

Cloud path

ESP32 → Wi-Fi → HTTPS/REST + JSON → Flask API → SQLite → browser dashboard. In the opposite direction, the dashboard submits commands; the backend queues them; the ESP32 polls, executes and reports events/acknowledgements.

Main software building blocks

- Arduino IDE / ESP32 support: firmware compilation and upload.
- SSD1306 + GFX: OLED graphics and pages; BH1750 / DHT libraries: sensor interfaces.
- DFRobotDFPlayerMini: serial control of audio playback.
- Wi-Fi / HTTPS / NTP: network connection, cloud transport and India-time greetings.
- Python + Flask: HTTP API; SQLite: readings, events, commands and device state.
- HTML / CSS / JavaScript: browser dashboard.

Timing values documented in the report

- Sensor update: approximately every 2.5 seconds.
- Telemetry upload: approximately every 5 seconds.
- Remote-command polling: approximately every 1.8 seconds.
- Condition-audio cooldown: approximately 5 minutes, helping prevent repetitive speech.

Why local + cloud?

Local processing keeps essential sensing and feedback responsive if Wi-Fi or the server is unavailable. The cloud adds remote visibility, history and remote commands; it should not be required for every sensor read.

4. Viva questions — short, defensible answers

Q. What is an embedded system?
A dedicated computing system that reads inputs and controls outputs for a defined purpose. The ESP32 runs PlantPal's sensing, interpretation and feedback logic.

Q. What is IoT?
Physical devices collect or exchange data over a network. PlantPal sends telemetry to a web service and accepts queued remote commands.

Q. Why ESP32?
It combines microcontroller capability with 2.4 GHz Wi-Fi and GPIO/peripheral interfaces for sensors, OLED, UART audio and buzzer.

Q. What is ADC and why GPIO34?
An ADC converts analog voltage to a digital reading. The soil sensor's AOUT is analog, and GPIO34 is an input-only ADC-capable pin.

Q. What is I²C?
A two-wire synchronous bus: SDA carries data and SCL carries clock. The OLED and BH1750 share GPIO21/22.

Q. What does BH1750 measure?
Ambient illuminance in lux. It does not measure photosynthesis or prove plant health by itself.

Q. What does DHT11 measure?
Air temperature and relative humidity. It is low-cost and modest in accuracy, so readings are indicative.

Q. Why a capacitive soil sensor?
It estimates moisture from an electrical response and is generally less prone to electrode corrosion than exposed resistive probes. Calibration is still necessary.

Q. How is soil percentage calculated?
Firmware maps raw readings between configured dry and wet references. Initial values are 3000 dry and 1200 wet; verify these in the final soil.

Q. What is UART?
An asynchronous serial interface. UART2 on GPIO16/17 controls the DFPlayer Mini.

Q. Why store audio on microSD?
The DFPlayer plays local MP3s without Internet streaming. Firmware selects numbered tracks for conditions, greetings and system events.

Q. What are REST and HTTPS?
REST-style HTTP endpoints exchange structured resources; HTTPS adds TLS protection when configured correctly. PlantPal exchanges JSON telemetry and commands.

Q. Why Flask and SQLite?
Flask provides a lightweight Python API; SQLite stores readings, events, commands and device state without a separate database server.

Q. What does HTTP 201 mean?
It generally indicates a request successfully created a resource. The report records HTTP 201 as a telemetry-upload success response.

Q. Does PlantPal water automatically?
No. It monitors and communicates. Watering is human-controlled; no pump or irrigation actuator is installed.

Q. What is the limitation of the health score?
It is a composite project indicator based on configured rules, not a scientifically validated diagnosis.

Q. What if the Internet fails?
Local sensing and interfaces are designed to continue. Cloud uploads and remote commands are unavailable until connectivity returns.

Q. What would you improve?
Calibrate moisture, verify light sensing in the final enclosure, use more accurate sensors, add authenticated cloud access, design a PCB and consider OTA updates.

5. Final integrated test plan

The report’s page 13 records earlier checks for OLED, soil readings, touch, DFPlayer audio and cloud telemetry. It also explicitly flags BH1750 at 0.0 lx and soil raw 3291 / 0% under initial calibration. Run the final assembled-system checks and only claim results actually observed.

Before power-on

- Check loose wires, shorts, exposed conductors and moisture near electronics. Keep electronics dry.
- Confirm common ground between ESP32 and external 5 V DFPlayer supply.
- Confirm speaker connects only to SPK1/SPK2; use moderate volume.
- Check soil sensor connector stays above soil; DHT11 is exposed to air; BH1750 has clear light exposure.
- Use a 2.4 GHz Wi-Fi hotspot if needed and verify local credentials.

Live test checklist

☐ ESP32 boots without repeated resets or brownout messages.

☐ OLED starts and shows readable status/sensor pages.

☐ Soil raw value changes plausibly between air/dry soil and moist soil; record readings before calibration.

☐ BH1750 changes between shade and bright light; do not assume 0.0 lx is correct without testing.

☐ DHT11 shows plausible room temperature and humidity; allow settling time.

☐ Touch module registers and triggers the intended response.

☐ DFPlayer plays a known microSD track at safe volume.

☐ Condition audio does not repeat continuously; cooldown/state-change logic behaves sensibly.

☐ Telemetry reaches the dashboard with a fresh timestamp.

☐ Send one remote command (e.g., CHECK_PLANT or PLAY_AUDIO) and verify execution.

☐ If safe, briefly disable Wi-Fi and verify local sensing/UI continue.

☐ Restore normal settings; ensure OLED/audio are enabled for the demo.

A clear 90-second demonstration

1. Introduce the problem (10 s). 2. Point out the sensors (15 s). 3. Show live readings and a touch interaction (15 s). 4. Play a known voice track (10 s). 5. Show fresh readings on the dashboard (20 s). 6. Execute one remote command if the network is ready (10 s). 7. Close with local-first design and human-controlled watering (10 s).

If something fails

- Sensor value stuck: check VCC/GND, signal pin and connector.
- BH1750 stays at 0 lux: check I²C wiring, power, sensor exposure and address; be honest if unresolved.
- Soil reads 0%: record raw value and recalibrate dry/wet references in the final soil; do not overwater to force a number.
- Dashboard stale: check Wi-Fi, API response and last-seen timestamp; local feedback may still work.
- No audio: check 5 V, common ground, SD folder/file numbering, UART wiring and audio-enabled state.

Team hand-off

Agree who explains the concept, who demonstrates hardware/sensors and who shows the dashboard. All three should know the pin map, local-vs-cloud distinction, calibration limitation and why watering remains manual.

Source basis: PlantPal Project Report (15 pages), especially pp. 7–13 for architecture, pin map, implementation and recorded validation checks. The development snapshot is not proof of tomorrow’s final integrated test.